

Python Programmation

CM 3 : Functions and Strings

Julien Bezançon

julien.bezancon@univ-lorraine.fr / <https://julienbez.github.io>



1 Before we start...

2 Your First Functions

3 Parameters and Arguments

4 Variadic Arguments

5 Python Scope

What did you learn during last lecture ?
Any questions ?

Doggy bag

Data structures

- Lists
- Dictionaries
- Tuples
- Sets

Advanced Python Stuffs

- Advanced loops
- Functions



A quick word about TDs

Last TD

- Not graded
- 1 week before delivery

Next TDs

- Might be graded
- To send at the end of the session !

Not doing all exercises is ok.
Do what you can.

- 1 Before we start...
- 2 Your First Functions**
- 3 Parameters and Arguments
- 4 Variadic Arguments
- 5 Python Scope

A Simple Function

```
def add_values(param1, param2):  
    value = param1 + param2  
    return value
```

```
add_values(2,4) # => 6
```

- Called by using its name
- Any number of parameters
- Parameters have no explicit type
- Returns "None" if no "return" statement

A Function that Returns Multiple Values

```
def add_values(param1, param2):  
    value1 = param1 + param2  
    value2 = param1 - param2  
    value3 = param1 * param2  
    return value1, value2, value3
```

```
add_values(2,4) # => 6, -2, 8
```

- Can return multiple values
- In that case, "return" works as a tuple

A Function with a Docstring

```
def add_values(param1, param2):  
    """  
    Add the values of param1 and param2.  
  
    Args: param1 (int), param2 (int)  
    Returns: value (int)  
    """  
    value = param1 + param2  
    return value
```

- The first string literal inside a function body is a docstring
- A docstring is optional but highly recommended
- Used to describe what the function does

- 1 Before we start...
- 2 Your First Functions
- 3 Parameters and Arguments**
- 4 Variadic Arguments
- 5 Python Scope

Basic Parameters

```
def add_values(param1, param2):  
    value = param1 + param2  
    return value
```

```
add_values(2,4) # => 6
```

```
add_values()
```

```
# => TypeError: add_values() missing 2 required positional  
#           arguments: 'param1' and 'param2'
```

- This function has two parameters
- These parameters are mandatory, they must be **specified**
- You can add any number of parameters

Default/Named Parameters

```
def add_values(param1,param2,multiplier=5):  
    value = (param1 + param2) * multiplier  
    return value
```

```
add_values(2,4,multiplier=5)      # => 30  
add_values(2,4)                   # => 30  
add_values(2,4,multiplier=10)     # => 60  
add_values(2,4,10)                # => 60
```

- You can add a default value to a parameter
- If a parameter is not specified, its default value is used (if it has one)

Arguments

```
def ask_yn(prompt, retries=4, complaint='Enter Y/N!'):
    for i in range(retries):
        ok = input(prompt)

        if ok in ('Y', 'y'):
            return True
        if ok in ('N', 'n'):
            return False

    print(complaint)
    return False
```

- Mandatory argument: "prompt"
- Keyword argument: "retries" and "complaint"

Arguments

```
# ask_yn(prompt, retries=4, complaint='...')
```

```
# Call with only the mandatory argument
```

```
ask_yn('Really quit?')
```

```
# Call with one keyword argument
```

```
ask_yn('OK to overwrite the file?', retries=2)
```

```
# Call with one keyword argument - in any order!
```

```
ask_yn('Update status?', complaint='Just Y/N')
```

```
# Call with all of the keyword arguments
```

```
ask_yn('Send text?', retries=2, complaint='Y/N please!')
```

The Dead Parrot Example

```
def parrot(voltage, state='a stiff', action='vroom',
           type='Norwegian Blue'):
    print("-- This parrot wouldn't", action, end=' ')
    print("if you put", voltage, "volts through it.")
    print("-- Lovely plumage, the", type)
    print("-- It's", state, "!")
```

- What does this function do ?

Dead Parrot: Valid Calls

```
# def parrot(voltage, state='...', action='...', type='...')

# 1 positional argument
parrot(1000)
# 1 keyword argument
parrot(voltage=1000)
# 2 keyword arguments
parrot(voltage=1000000, action='VOOOOOM')
# 2 keyword arguments
parrot(action='VOOOOOM', voltage=1000000)
# 3 positional arguments
parrot('a million', 'bereft of life', 'jump')
```

Dead Parrot: Invalid Calls

```
# def parrot(voltage, state='...', action='...', type='...')

# required argument missing
parrot()

# non-keyword argument after a keyword argument
parrot(voltage=5.0, 'dead')

# duplicate value for the same argument
parrot(110, voltage=220)

# unknown keyword argument
parrot(actor='John Cleese')
```

Other Examples of Keyword Arguments

```
print("hello", sep=' ', end='\n', file=sys.stdout, flush=False)
range(start, stop, step=1)
enumerate(iter, start=0)
int(x, base=10)
seq.sort(*, key=None, reverse=None)
```

```
subprocess.Popen(args, bufsize=-1, executable=None,
stdin=None, stdout=None, stderr=None, preexec_fn=None,
close_fds=True, shell=False, cwd=None, env=None,
universal_newlines=None, startupinfo=None,
creationflags=0, restore_signals=True,
start_new_session=False, pass_fds=(), encoding=None,
errors=None, text=None)
```

A Weird Case: Mutable default parameter values

```
def my_func(my_list = []):  
    my_list.append("hello")  
    return my_list
```

```
my_func([1,2,3]) # => [1,2,3,"hello"]  
my_func() # => ["hello"], but what happens if I repeat this line ?  
my_func() # => ?  
my_func() # => ?
```

- What happen if I repeat "my_func()" several times ?
- Why does this happen ?

A Weird Case: Explanation

```
def my_func(my_list = []):  
    print(id(my_list))  
    my_list.append("hello")  
    return my_list
```

```
my_func() # => 140095566958408, ["hello"]  
my_func() # => 140095566958408, ["hello", "hello"]  
my_func() # => 140095566958408, ["hello", "hello", "hello"]
```

- Default parameters values are defined only once, at function definition
- Each new call points to the same object

- 1 Before we start...
- 2 Your First Functions
- 3 Parameters and Arguments
- 4 Variadic Arguments**
- 5 Python Scope

Variadic Positional Arguments

```
def product(*nums, scale=1):  
    p = scale  
    for n in nums:  
        p *= n  
    return p
```

```
product(3,5) # => 15
```

```
product(3,4,2) # => 24
```

```
product(3,5,scale=10) # => 150
```

- A parameter of form "*arg" captures excess arguments
- These arguments are stored into the "arg" tuple
- Parameters after "*args" must be default/named ones !

```
print(*objects, sep=' ', end='\n', file="toto.txt", flush=False)
```

Unpacking Variadic Positional Arguments

```
def product(*nums, scale=1):  
    p = scale  
    for n in nums:  
        p *= n  
    return p
```

```
numbers = [1,2,3,4,5,6]  
product(*numbers) # equivalent to product(1,2,3,4,5,6)
```

- The syntax `"*seq"` unpack a sequence into its constituent components

Variadic Keyword Arguments

```
def authorize(quote, **speaker_info):  
    print(">", quote)  
    print("-" * (len(quote) + 2))  
    for k, v in speaker_info.items():  
        print(k, v, sep=': ')
```

```
authorize("If music be the food of love, play on.",  
         playwright="Shakespeare", act=1, scene=1,  
         speaker="Duke Orsino")
```

- A parameter of form "**kwargs" captures all excess keyword arguments
- These arguments are stored into the "kwargs" dictionary

Unpacking Variadic Keyword Arguments

```
authorize("If music be the food of love, play on.",  
          playwright="Shakespeare", act=1, scene=1,  
          speaker="Duke Orsino")
```

```
# > If music be the food of love, play on.  
# -----  
# act: 1  
# scene: 1  
# speaker: Duke Orsino  
# playwright: Shakespeare
```

Variadic Keyword Arguments

```
info = {  
    'sonnet': 18,  
    'line': 1,  
    'author': "Shakespeare"  
}
```

```
authorize("Shall I compare thee to a summer's day", **info)
```

```
# > Shall I compare thee to a summer's day  
# -----  
# line: 1  
# sonnet: 18  
# author: Shakespeare
```

- 1 Before we start...
- 2 Your First Functions
- 3 Parameters and Arguments
- 4 Variadic Arguments
- 5 Python Scope**

Variables in Functions

```
def add_values(param1, param2):  
    value = param1 + param2  
    return value  
  
print(value) # => NameError: name 'value' is not defined.
```

Variables in Functions

```
def add_values(param1, param2):  
    value = param1 + param2  
    return value  
  
print(value) # => NameError: name 'value' is not defined.
```

- Variables defined inside a function only work in said function
- A variable is only available from inside the region it is created

The LEGB rule

When Python look up variable names, it does it in this order :¹

1. Local - inside the current function
2. Enclosing - inside enclosing functions
3. Global - at the top level of the module
4. Built-in - in Python's built-in namespace

¹https://www.w3schools.com/python/python_scope.asp

Local Scope

```
def add_values(param1, param2):  
    value = param1 + param2  
    return value
```

```
print(value) # => NameError: name 'value' is not defined.
```

- "value" is only available in the "add_values" function
- "value" does not exist outside of the "add_values" function

Enclosing Scope

```
def add_values(param1, param2):  
    value = param1 + param2  
    def show_value():  
        print(value)  
    show_value()
```

```
add_values(2,4) # => 6
```

- "value" can be accessed from a function within the "add_values" function
- Why? Because "show_value" is **inside** the "add_values" function

Global Scope

```
value = 300
```

```
def show_value():  
    print(value)
```

```
show_value() # => 300
```

- Variables created outside of a function can be used by anyone
- Variables created outside of a function == global scope

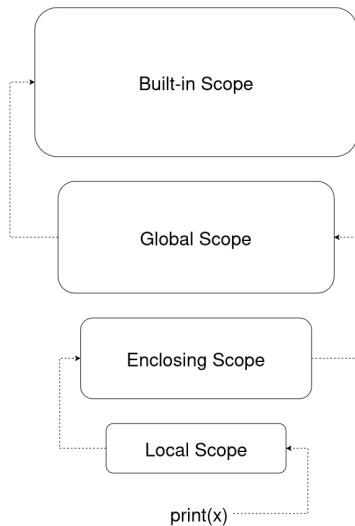
Built-in Scope

Built-ins are always available and have an usage

- `int()`, `bool()`, `str()`, `float()`, etc
- `print()`, `zip()`, `input()`, etc
- `sorted()`, `min()`, `max()`, `len()`, etc

Built-ins are always available.

Scopes Summary



Python checks scopes in this order :

1. Local scope
 2. Enclosing scope
 3. Global scope
 4. Built-in scope
- If "x" not found, return an error

Almost Everything is Global Scope

```
success = True

if success:
    desc = "winner!"
else:
    desc = "loser"

print(desc)
```

- Only functions may define a new scope (local or enclosing)
- Other statements and loops does not define new scopes

